$ ./linux2_nexo.sh

LINUX II

NEXO · ADMINISTRACIÓN AVANZADA

- POSIX Avanzado · Pipes · Filtros
- SSH y Administración Remota
- Shell Scripting Profesional
- Puente hacia Linux III (systemd)

CTT · Técnico en Redes y Software · Prof. Juan Damián Pajares García · Semana 1–16



HOJA DE RUTA

Linux I

Fundamentos · CLI · FHS

Linux II

ACTUAL

POSIX Avanzado · SSH · Scripting

Linux III

systemd · Servicios · Hardening

¿Qué aprendimos en Linux I?

Antes de avanzar — activación de conocimientos previos

Kernel + GNU/Linux

Linux = Kernel (Torvalds) + herramientas GNU (Stallman). No es un SO monolítico, es un ecosistema modular.

Filosofía Unix

Hacer una cosa bien · Todo es archivo · Texto plano · Pipes. El ADN técnico que gobierna cada comando.

CLI y FHS

La terminal como interfaz de poder. Jerarquía /etc /home /var /usr /proc — dónde vive cada cosa del sistema.

Comandos Esenciales

pwd ls cd mkdir touch cp mv rm cat grep · La base sin la cual no se puede administrar ningún sistema.

» ½ ™ ™ G P @ I O @ J I > @ K O P < G œ @ I J Q < O J < < ? H D I D N O M < ? J M

SSH: Criptografía Asimétrica en la Práctica

Secure Shell — el protocolo estándar para administración remota segura



Comandos SSH Esenciales

Conectar:	`$ ssh usuario@192.168.1.10`
Puerto custom:	`$ ssh -p 2222 user@host`
Generar par de claves:	`$ ssh-keygen -t ed25519 -C 'ctt@lab'`
Copiar clave pública:	`$ ssh-copy-id user@servidor`
Transferir archivo:	`$ scp archivo.txt user@host:/destino/`
Túnel local (port fwd):	`$ ssh -L 8080:localhost:80 user@host`

Hardening sshd_config

```
Port 2222              # No usar 22
PermitRootLogin no
PasswordAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
AllowUsers admin sysop
```

Estructura Canónica de un Script Bash

El estándar profesional que distingue un script de producción de un experimento

bash · /usr/local/bin/monitor.sh

```bash
#!/usr/bin/env bash
# ——— Encabezado obligatorio ——————
set -euo pipefail  # Exit on error, undef var, pipe fail
readonly VERSION='1.0.0'  # Constantes en UPPER_CASE
LOG='/var/log/monitor.log'

log() { echo "[$(date)] $*" >> $LOG; }  # Función de logging
check_service() {
  systemctl is-active --quiet "$1" \
    && log "OK: $1" \
    || log "FAIL: $1"
}

main() {
  for svc in nginx ssh docker; do  # Iterar servicios
    check_service "$svc"
  done
}
```

set -euo pipefail

La directiva de seguridad más importante. Aborta el script ante cualquier error, variable no definida o falla en pipe.

Funciones modulares

Todo script profesional encapsula lógica en funciones. main() como punto de entrada explícito — patrón obligatorio.

cron — Automatización

Programar la ejecución: '*/5 * * * * /usr/local/bin/monitor.sh' — cada 5 minutos, sin

MIN HORA DIA MES DOW COMANDO

Desde Scripts hacia Servicios Administrados

Cómo Linux II construye la base conceptual para Linux III

01

Script Bash manual

Ejecutado a mano en terminal. Depende del usuario. No persiste entre reinicios.

```
$ bash /usr/local/bin/mi_app.sh
```

Linux I + II

02

Cron + Script

Ejecución programada. Aún sin manejo de errores del sistema ni integración con el OS.

```
*/5 * * * *
/usr/local/bin/mi_app.sh >>
/var/log/app.log
```

Linux II · U3

03

Systemd Unit File (.service)

El script se convierte en ciudadano de primer nivel. Restart automático, logging centralizado, seguridad con cgroups.

```
[Service]
ExecStart=/usr/local/bin/mi_app.s
h
Restart=always
NoNewPrivileges=true
```

Linux III

Hoja de Ruta Linux II

16 semanas · 3 Unidades · Dos perfiles de aprendizaje

U1 — Comandos Avanzados y POSIX

Sems 1–5 · ~15 hs

- Redirecciones: >, >>, <, 2>, &>
- Pipes: encadenamiento de comandos
- grep -E, awk, sed, cut, sort, uniq
- find + xargs — búsquedas avanzadas
- Procesos: ps, kill, htop, lsof
- systemctl + journalctl (intro)

U2 — SSH y Administración Remota

Sems 6–11 · ~18 hs

- Criptografía asimétrica: pub/priv key
- ssh-keygen ed25519, ssh-copy-id
- SCP, SFTP — transferencia segura
- Hardening de sshd_config
- Port forwarding y túneles SSH
- Fail2ban + auditoría de accesos

U3 — Shell Scripting y Automatización

Sems 12–16 · ~15 hs

- Estructura profesional: shebang, set
- Variables, $@, $#, getopts
- Condicionales, loops, funciones
- Manejo de errores y exit codes
- cron — programación de tareas
- Script de monitoreo integrador

CTT · Prof. Juan Damián Pajares García

Laboratorio Inicial — Diagnóstico de Saberes

Abrir la terminal. Hoy medimos desde dónde partimos.

PERFIL A

Sin experiencia previa

01 Diagnóstico básico

```
$ cat /etc/os-release && uname -r
```

02 Listar archivos de config

```
$ ls /etc | grep -E '\.conf$' | wc -l
```

03 Auditar usuarios del sistema

```
$ awk -F: '$3>=1000 {print $1}' /etc/passwd
```

04 Ver log del sistema

```
$ tail -20 /var/log/syslog
```

05 Pipeline integrador

```
$ ps aux | grep -v grep | awk '{print $1}' | sort | uniq -c | sort -rn | head -5
```

PERFIL B

Con experiencia en Bash o Python

01 Script de auditoría del sistema

```
$ set -euo pipefail
$ awk -F: ... | sed ... | sort -rn
```

02 Reporte de procesos por usuario

```
$ ps -eo user,pid,pcpu,pmem,comm | awk...
```

03 Generar informe en archivo

```
$ exec > /tmp/audit_$(date +%F).log 2>&1
```

04 Verificar servicios activos

```
$ systemctl list-units --type=service | awk...
```

05 Desafío ⭐ — Script + cron

```
# Combinar todo en un script
# Programar con crontab -e
```

CIERRE DE CLASE NEXO

Linux II

Administración Avanzada

Lo que construimos hoy:

- ✓ Consolidación de Linux I — base sólida
- ✓ Mapa curricular completo de las 16 semanas
- ✓ Flujos estándar POSIX (stdin/stdout/stderr)
- ✓ El trío de filtros: grep · awk · sed
- ✓ Pipes y redirecciones — composición de comandos
- ✓ Vista previa de SSH, Scripting y Linux III

```
$ echo "¡Hasta la próxima clase!"  🐧
```

Próximas Clases

Sem 2–5
POSIX Profundo
awk/sed avanzado · procesos · find+xargs

Sem 6–11
SSH + Hardening
Claves asimétricas · sshd_config · fail2ban

Sem 12–16
Shell Scripting
Scripts de producción · cron · monitoring

Linux III
systemd completo
Servicios · journald · cgroups · sandboxing